HTML Phase – Master Development & QA Checklist

Applicable to every HTML website/project, regardless of design, client, industry, or project type.

---

1. Section Heading Structure

- All section titles must use a common reusable heading wrapper/component.
- Do not style headings solely based on h2, h3, h4, etc.
- Use a reusable structure:

```
<div class="section-heading">
   <span class="section-heading__eyebrow">Beyond the Ocean</span>
   <h2 class="section-heading__title">Who We Are</h2>
</div>
```

- Heading styling must continue to work if h2 changes to h3, h4, etc.
- Semantic heading levels must be selected according to content hierarchy.
- Visual appearance must be controlled by CSS/component classes.
- Do not use heading tags simply because they have the desired default browser styling.

Rule:

> HTML determines meaning. CSS determines appearance.

---

2. Section Height

- Avoid fixed heights for content-heavy sections.
- Do not use `min-height: 100vh` unless a true full-screen section is required.
- Avoid unnecessary `max-height`.
- Avoid fixed `height` values that can cause content clipping.
- Allow sections to grow naturally with content.
- Prefer:

```
section {
   padding-block: clamp(3rem, 6vw, 7rem);
}
```

- Check that sections do not become unnecessarily tall on large screens.
- Check that content does not become compressed or clipped on smaller screens.

---

3. Responsive Section Spacing

- Avoid excessive fixed `padding-top`.
- Avoid excessive fixed `padding-bottom`.
- Use responsive spacing with `clamp()`.
- Check spacing at different viewport widths.
- Check spacing at different browser zoom levels.
- There must be **no unexplained large gap above or below any section**.

Particular attention must be given when:

- Browser is zoomed out.
- Browser is zoomed in.
- Screen is unusually wide.
- Screen is a smaller laptop.
- Content length changes.

---

4. Browser Zoom Testing

Every page must be checked at:

- 75%
- 80%
- 90%
- 100%
- 110%
- 125%
- 150%

Check specifically for:

- Massive top gaps.
- Massive bottom gaps.
- Excessive section heights.
- Content being pushed away from headings.
- Images creating whitespace.
- Broken layouts.
- Overlapping elements.
- Text clipping.
- Horizontal scrolling.
- Navigation problems.
- Incorrect hero height.
- Broken animations.
- Content disappearing.

---

5. Large Desktop Testing

Test on large desktop resolutions.

Verify:

- Sections do not become excessively tall.
- No huge empty areas.
- Content remains visually balanced.
- Images remain correctly positioned.
- Hero section remains proportional.
- Text does not become excessively separated from other content.
- Grid/card layouts remain balanced.

---

6. Small Laptop Testing

Small laptop screens must receive special attention.

Verify:

- No horizontal overflow.
- No horizontal lines appearing because of broken widths.
- No content clipping.
- No overlapping elements.

- Navigation remains functional.
- Buttons remain accessible.
- Cards remain inside their containers.
- Images remain correctly positioned.
- Text wraps correctly.
- Hero sections remain usable.
- Footer does not break.
- Forms remain usable.
- No breakpoint-related layout failures occur.

---

7. Responsive Device Testing

Test at minimum:

- Large desktop
- Desktop
- Small laptop
- Tablet landscape
- Tablet portrait
- Large mobile
- Small mobile

Do not test only at the standard desktop/mobile breakpoints.

Intermediate widths must also be checked.

---

8. Section Layout

Every section should:

- Use a common `.container` / `.wrap`.
- Determine its height naturally from its content.
- Use Flexbox or Grid where appropriate.
- Avoid unnecessary absolute positioning.
- Avoid unnecessary fixed dimensions.
- Prevent content overflow.
- Prevent images from unexpectedly increasing section height.
- Maintain consistent alignment.

- Maintain consistent spacing.

---

9. Reusable Components

Identify repeated UI patterns and create reusable classes/components for:

- Section headings
- Buttons
- Cards
- Containers
- Forms
- Form fields
- Navigation
- Breadcrumbs
- Accordions
- Tabs
- CTA sections
- Testimonials
- Blog cards
- Footer elements
- Social icons

Do not duplicate the same CSS unnecessarily.

---

10. Accordion Behaviour

All accordions/FAQs must be tested.

Mandatory behaviour:

- When one accordion opens, the previously opened accordion must automatically close.
- Only one accordion should remain open at a time unless the approved design explicitly requires multiple open items.
- Open/close animation must be smooth.
- Content must not be clipped.
- Accordion height must adapt to its content.
- No fixed heights that cut off content.
- Keyboard interaction must work.

- Active state must be clearly visible.
- Accordion must work correctly on mobile.
- Accordion must continue working after browser resizing.

---

11. Navigation

Check:

- Desktop navigation.
- Mobile navigation.
- Hamburger menu.
- Dropdown menus.
- Nested menus.
- Active states.
- Hover states.
- Keyboard navigation.
- Mobile menu closing behaviour.
- No menu overflow.
- No menu content appearing outside the viewport.
- No broken links.
- No duplicate navigation items.

---

12. Images & Media

- Correct image is used.
- Correct image dimensions/aspect ratio.
- No distorted images.
- No unnecessary oversized images.
- Proper `object-fit`.
- Proper `object-position`.
- Responsive image behaviour.
- Images do not create unexpected whitespace.
- Images do not overflow containers.
- Images have appropriate `alt` attributes.
- Decorative images should use appropriate empty `alt=""`.
- Videos must not cause layout problems.
- Background videos must remain responsive.

---

13. Typography

Check:

- Correct font family.
- Correct font weights.
- Correct font sizes.
- Correct line heights.
- Correct letter spacing.
- Responsive heading sizes.
- Responsive paragraph sizes.
- No text overflow.
- No awkward heading breaks.
- No text clipping.
- No excessive line lengths.
- Fonts load correctly.
- No major layout shift caused by font loading.

---

14. No Horizontal Overflow

Every page must be checked for:

- Horizontal scrollbar.
- Elements extending outside viewport.
- Oversized images.
- Long text.
- Long URLs.
- Absolute-positioned elements.
- Animations extending beyond viewport.
- Mobile navigation overflow.
- Cards exceeding container width.
- Forms exceeding viewport width.

Mandatory result:

> No unintended horizontal scrolling anywhere on the website.

---

15. Accessibility

Verify:

- Semantic HTML.
- Correct heading hierarchy.
- Image `alt` text.
- Form labels.
- Keyboard accessibility.
- Visible focus states.
- Accessible buttons.
- Accessible links.
- Accessible accordions.
- Accessible navigation.
- Sufficient colour contrast.
- No interaction dependent solely on colour.
- Interactive elements are usable without a mouse.

---

16. Animation & Motion

Use modern motion techniques where appropriate.

Available design/development resources should be considered, including:

- Ponytail
- Motion Skill
- UIUXProMax
- 21st.dev
- Graphify
- Framer Motion
- GSAP
- ScrollTrigger
- Modern CSS transitions/animations

However:

- Do not add animation unnecessarily.
- Animations must not cause layout shifts.
- Animations must not cause horizontal overflow.
- Animations must not block interaction.
- Scroll animations must work at different screen sizes.

- Motion must remain smooth.
- Reduced-motion preferences should be respected where appropriate.

---

17. Security Headers

All 4 required security headers must be present.

Verify the production/staging server response headers and ensure the project has the required security protections.

At minimum, check:

1. Content-Security-Policy

Content-Security-Policy

2. Strict-Transport-Security

Strict-Transport-Security

3. X-Content-Type-Options

X-Content-Type-Options: nosniff

4. X-Frame-Options

X-Frame-Options

Do not merely add them inside HTML `<meta>` tags where the header needs to be an actual HTTP response header.

Verify the actual server response using an appropriate security/header scanner or browser/network inspection.

Also check for:

- HTTPS.
- No mixed content.
- No insecure HTTP resources.
- No unnecessary server information exposure.
- No exposed development/debug information.
- Secure external resource loading.

---

18. HTML Validator

Run the completed HTML through an HTML validator.

Mandatory requirement:

> **0 HTML validation errors.**

Check:

- Invalid HTML.
- Incorrect nesting.
- Missing required attributes.
- Duplicate IDs.
- Invalid attributes.
- Broken elements.
- Incorrect heading structure.
- Unclosed tags.
- Invalid ARIA usage.
- Invalid links.
- Deprecated/invalid markup where applicable.

Warnings should also be reviewed and resolved where practical.

---

19. CSS Validation / Code Quality

Check CSS for:

- Invalid CSS.
- Duplicate rules.
- Unnecessary overrides.
- Excessive specificity.
- Unused styles where identifiable.
- Broken media queries.
- Conflicting styles.
- Fixed heights causing layout problems.
- Excessive `!important`.
- Duplicate component styles.
- Unnecessary absolute positioning.

---

20. JavaScript Errors

Mandatory requirement:

> **Zero console errors.**

Open browser DevTools and check:

- Console errors.
- JavaScript exceptions.
- Failed imports.
- Failed API requests.
- Missing files.
- Broken image requests.
- Failed font requests.
- CORS errors.
- Mixed-content errors.
- Failed third-party scripts.
- Unhandled promise errors.

No unresolved critical console errors should remain.

Warnings should also be reviewed rather than blindly ignored.

---

21. Broken Links

Check all:

- Navigation links.
- Footer links.
- CTA links.
- Buttons.
- Social links.
- Internal page links.
- External links.
- Breadcrumbs.
- Logo links.
- Mobile navigation links.

Mandatory:

No unintended broken links.

---

22. Forms

Test:

- Required fields.
- Validation.
- Email validation.
- Phone validation.
- Error messages.
- Success messages.
- Submit button.
- Loading state.
- Duplicate submission prevention.
- Mobile usability.
- Keyboard navigation.
- Form reset behaviour.
- Backend/API response where applicable.
- Spam protection where required.

---

23. SEO / Staging Protection

For development/staging websites:

Robots meta

<meta name="robots" content="noindex, nofollow, noarchive">

robots.txt

Where appropriate:

User-agent: *
Disallow: /

Verify that the staging website is not accidentally indexed.

Before production deployment:

- Remove staging `noindex`.
- Review `robots.txt`.
- Enable crawling where intended.
- Verify canonical URLs.
- Verify sitemap.
- Verify production metadata.

Never accidentally leave production blocked from search engines.

---

24. PageSpeed Insights

Run the website through **Google PageSpeed Insights**.

Mandatory target:

> **Performance Score: 100**

Check both:

- Mobile
- Desktop

Also review:

- LCP
- CLS
- INP
- FCP
- TTFB
- Render-blocking resources
- Image optimization
- JavaScript execution
- CSS optimization
- Font loading
- Caching
- Third-party scripts

A visually complete website is **not considered HTML-phase complete if performance has not been checked**.

---

25. Core Web Vitals

Verify:

- Largest Contentful Paint
- Cumulative Layout Shift
- Interaction to Next Paint
- First Contentful Paint
- Total Blocking Time where applicable to the report
- Speed Index
- Time to First Byte

Pay particular attention to:

- Hero images.
- Hero videos.
- Web fonts.
- Large JavaScript bundles.
- Animation libraries.
- Third-party scripts.
- Lazy loading.
- Layout shifts.

---

26. Performance Optimization

Before final delivery:

- Compress images.
- Use modern image formats where appropriate.
- Avoid unnecessarily huge images.
- Lazy-load below-the-fold images.
- Avoid unnecessary JavaScript.
- Minimize blocking resources.
- Optimize CSS.
- Optimize font loading.
- Avoid unnecessary libraries.
- Avoid loading libraries that are not used.
- Optimize animations.
- Avoid excessive DOM complexity.
- Check network requests.

---

27. Console / Network QA

DevTools must be checked for:

Console

- 0 errors.
- No unresolved critical warnings.

Network

- No unexpected 404s.
- No 500 errors.
- No failed JS files.
- No failed CSS files.
- No missing fonts.
- No missing images.
- No blocked critical resources.
- No mixed-content requests.

---

28. Footer

Verify:

- Footer layout.
- Footer responsiveness.
- Footer links.
- Social links.
- Copyright.
- Contact information.
- Mobile layout.
- No overflow.
- Correct spacing.

For InterSmart footer credit, always use:

D:\footerintersmart-logo.svg

Credit should use the approved design:

Designed and Developed by InterSmart

Do not recreate or substitute the logo.

---

29. Browser Compatibility

Check the website on supported modern browsers, particularly:

- Google Chrome
- Microsoft Edge
- Firefox
- Safari where applicable

Check:

- Layout.
- Fonts.
- Animations.
- Navigation.
- Forms.
- Accordions.
- Responsive behaviour.
- JavaScript functionality.

---

30. WordPress/CMS Conversion Readiness

The HTML structure must be suitable for later WordPress/CMS conversion.

Ensure:

- Reusable components.
- Logical section structure.
- Clear class naming.
- Minimal duplicated markup.
- Reusable buttons.
- Reusable cards.

- Reusable headings.
- Reusable containers.
- Semantic HTML.
- Clean separation of content and presentation.
- No unnecessary inline styles.
- No excessive hardcoded content inside JavaScript.
- No layout dependent on fragile absolute positioning.

---

31. Final Visual QA

Compare the implementation against the approved design/reference.

Verify:

- Overall layout.
- Section order.
- Typography.
- Colours.
- Spacing.
- Images.
- Icons.
- Buttons.
- Cards.
- Borders.
- Shadows.
- Radius.
- Hover states.
- Active states.
- Animations.
- Responsive behaviour.

---

32. Final Responsive QA

Before marking the project complete, verify:

Desktop

- Layout correct.

- No gaps.
- No overflow.
- No clipping.

Large Desktop

- No excessive section heights.
- No huge whitespace.
- Content remains balanced.

Small Laptop

- No responsive issues.
- No broken lines.
- No clipping.
- No horizontal scrolling.
- No overlapping content.

Tablet

- Layout adapts correctly.
- Navigation works.
- Cards stack correctly.

Mobile

- No horizontal overflow.
- Text fits.
- Buttons fit.
- Images fit.
- Forms work.
- Navigation works.
- Accordions work.

Zoom

- 75%
- 80–90%
- 100%
- 110–125%
- 150%

No major layout issue should occur.

---

33. Mandatory Final Release Gates

The HTML phase **must not be marked complete** until all of the following are satisfied:

QA Gate	Requirement
HTML Validator	**0 errors**
Console	**0 errors**
Broken Resources	**0 critical failures**
Horizontal Overflow	**0 unintended overflow**
Security Headers	**All 4 present**
PageSpeed Mobile	**100 Performance target**
PageSpeed Desktop	**100 Performance target**
Responsive Desktop	**Pass**
Large Desktop	**Pass**
Small Laptop	**Pass**
Tablet	**Pass**
Mobile	**Pass**
Browser Zoom	**75–150% checked**
Section Gaps	**No abnormal top/bottom gaps**
Accordion	**Only one open at a time**
Images	**Correct proportions**
Forms	**Pass**
Navigation	**Pass**
Accessibility	**Pass**
Staging Robots	**Crawling/indexing disabled**
Production Robots	**Reviewed before launch**
Security	**Pass**

WordPress Readiness	**Pass**
Footer Credit	**Approved InterSmart SVG used**
Visual QA	**Pass**

---

34. Final HTML Phase Definition of Done

HTML Phase = COMPLETE only when:

> **Design is approved + HTML is semantically correct + reusable components are used + responsive layouts work across desktop/laptop/tablet/mobile + 75–150% browser zoom is checked + no abnormal section gaps exist + no horizontal overflow exists + accordions work correctly + HTML validation has 0 errors + console has 0 errors + required security headers are present + staging crawling/indexing is disabled + PageSpeed target is 100 + images/media are optimized + accessibility basics pass + no broken resources/links remain + footer uses the approved InterSmart SVG + code is clean and ready for WordPress/CMS conversion.**

This should be treated as the **standard mandatory HTML development and QA checklist for every project**, rather than a project-specific checklist.

also folder should bee sass html structre

Yes. Add a **mandatory project folder and Sass architecture requirement** as well. Here is the updated section you can append to the master checklist.

35. Mandatory HTML Project Folder Structure

Every HTML project must follow a clean, standardized folder structure.

Recommended structure:

```
project/
│
├── index.html
├── about.html
├── services.html
├── contact.html
│
├── assets/
```

```
│   ├── images/
│   ├── icons/
│   ├── videos/
│   ├── fonts/
│   └── documents/
│
├── css/
│   ├── main.css
│   └── main.min.css
│
├── scss/
│   ├── main.scss
│   │
│   ├── abstracts/
│   │   ├── _variables.scss
│   │   ├── _mixins.scss
│   │   ├── _functions.scss
│   │   └── _breakpoints.scss
│   │
│   ├── base/
│   │   ├── _reset.scss
│   │   ├── _typography.scss
│   │   └── _base.scss
│   │
│   ├── components/
│   │   ├── _buttons.scss
│   │   ├── _cards.scss
│   │   ├── _section-heading.scss
│   │   ├── _accordion.scss
│   │   ├── _forms.scss
│   │   ├── _navigation.scss
│   │   └── _footer.scss
│   │
│   ├── layout/
│   │   ├── _header.scss
│   │   ├── _container.scss
│   │   ├── _sections.scss
│   │   └── _footer.scss
│   │
│   ├── pages/
│   │   ├── _home.scss
│   │   ├── _about.scss
│   │   ├── _services.scss
│   │   └── _contact.scss
```

```
│   │
│   └── vendors/
│       └── _vendors.scss
│
├── js/
│   ├── main.js
│   └── components/
│       ├── accordion.js
│       ├── navigation.js
│       └── animation.js
│
└── README.md
```

Folder Structure Rules

- Do not keep everything in one huge CSS file during development.
- Use **SCSS/Sass** for styling.
- `main.scss` should act as the primary Sass entry point.
- Organize styles into:
    - Abstracts
    - Base
    - Components
    - Layout
    - Pages
    - Vendors
- Compile SCSS into CSS.
- HTML should load the compiled CSS, not depend on the SCSS source directly.
- Avoid duplicate styles between page-specific and component files.
- Common components must have their own SCSS files.
- Page-specific styles should remain inside the appropriate `pages/` file.
- Variables such as colours, typography, spacing, breakpoints and common dimensions should be centralized.
- Mixins should be reused instead of duplicating responsive CSS.
- Breakpoints should be centralized rather than randomly defined throughout the project.

---

36. Sass Architecture Rules

Use a consistent Sass architecture.

Variables

Centralize:

```
$primary-color: #...;
$secondary-color: #...;
$text-color: #...;
$container-width: ...;
$border-radius: ...;
```

Breakpoints

Do not scatter arbitrary media-query values throughout the project.

Use centralized breakpoint variables/mixins.

```
@include respond-to(tablet) {
    ...
}
```

Components

Repeated UI should have component-level SCSS:

```
components/
├── _buttons.scss
├── _cards.scss
├── _section-heading.scss
├── _accordion.scss
└── _forms.scss
```

Pages

Only page-specific styling should go into:

```
pages/
├── _home.scss
├── _about.scss
└── _contact.scss
```

Do not put global component styles inside page files.

---

37. HTML + Sass Separation

Maintain a clear separation:

HTML

↓
Semantic structure
↓
Reusable classes
↓
SCSS components
↓
Compiled CSS

Do not:

- Put large amounts of inline CSS in HTML.
- Duplicate the same styles across pages.
- Use `<style>` blocks unnecessarily.
- Use inline styles for normal component styling.
- Create page-specific versions of components unnecessarily.

---

38. Naming Convention

Use a consistent naming system such as BEM where appropriate.

Example:

```
<section class="about-section">
   <div class="container">
      <div class="section-heading">
         <span class="section-heading__eyebrow">
            About Us
         </span>

         <h2 class="section-heading__title">
            Who We Are
         </h2>
      </div>
   </div>
</section>
```

SCSS:

```
.section-heading {
   ...

   &__eyebrow {
```

```
    ...
  }

  &__title {
    ...
  }
}
```

This makes the HTML easier to maintain and later convert into WordPress components.

---

39. Final Code Organization Check

Before HTML phase completion:

- Folder structure follows the standard architecture.
- SCSS is separated from compiled CSS.
- `main.scss` is the primary Sass entry point.
- Variables are centralized.
- Mixins are centralized.
- Breakpoints are centralized.
- Components have reusable SCSS.
- Page-specific styles are separated.
- No unnecessary inline CSS.
- No unnecessary duplicate CSS.
- No giant unstructured stylesheet.
- HTML uses reusable classes.
- JavaScript is separated from HTML.
- Images/assets are organized.
- Fonts are organized.
- The project can be cleanly handed over for WordPress/CMS conversion.

Final principle

> **The HTML project must be designed as a maintainable frontend system, not just a collection of visually matching HTML pages.**

That means **clean folder structure + semantic HTML + reusable components + Sass architecture + responsive behaviour + security + performance + accessibility + zero validation/console errors + WordPress-ready structure**.